



Name: Rahin Ibne Harun

Id: 23-54177-3

Sec: U

Course: Compiler Design Assignment

Final Lab Task-5

Task 1:

Task 1: Write a C++ program that takes a **complete DFA** as input, constructs its **complement DFA**, and tests multiple input strings using the complement DFA. A complement DFA is created by changing every **final state** into a **non-final state**, and every **non final state** into a **final state**. The transition table remains the same.

Input Format

- The first line contains two integers n and m.
 - n = number of states, states are numbered from 0 to n-1
 - m = number of input symbols
- The next m lines contain the input symbols.
- The next n × m lines contain the DFA transition table in this format:
 - current_state input_symbol next_state
- The next line contains the start state.
- The next line contains an integer f, the number of final states.
- The next line contains f integers representing the final states.
- The next line contains an integer q, the number of test strings.
- The next q lines contain one input string each.

Output Format

Print:

- the final states of the complement DFA
- for each input string, print Accepted or Rejected

Sample Input	Sample Output
3 2 a b 0 a 1 0 b 0 1 a 1 1 b 2 2 a 1 2 b 0 0 1 2 4 ab a bbb aab	Complement Final States: 0 1 String: ab Rejected String: a Accepted String: bbb Accepted String: aab Rejected

Answer:

```

#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <set>

using namespace std;

int main() {
    int n, m;
    if (!(cin >> n >> m)) return 0;

    // Read input symbols
    vector<char> symbols(m);
    for (int i = 0; i < m; ++i) {
        cin >> symbols[i];
    }

```

```

// Transition table: current_state -> {symbol -> next_state}
// Using a 2D array/vector since states are 0 to n-1
// We use a map for symbols to handle non-integer character inputs
vector<map<char, int>> transitions(n);
for (int i = 0; i < n * m; ++i) {
    int curr, next;
    char sym;
    cin >> curr >> sym >> next;
    transitions[curr][sym] = next;
}

int start_state;
cin >> start_state;

int f;
cin >> f;
set<int> original_final_states;
for (int i = 0; i < f; ++i) {
    int state;
    cin >> state;
    original_final_states.insert(state);
}

// Create Complement Final States
// Logic: If state was NOT in original_final_states, it IS in complement_final_states
vector<int> complement_final_states;
for (int i = 0; i < n; ++i) {
    if (original_final_states.find(i) == original_final_states.end()) {
        complement_final_states.push_back(i);
    }
}

// Output Complement Final States
cout << "Complement Final States:" << endl;
for (int i = 0; i < complement_final_states.size(); ++i) {
    cout << complement_final_states[i] << (i == complement_final_states.size() - 1 ? "" : " ");
}

```

```

}
cout << "\n" << endl;

int q;
cin >> q;
while (q--) {
    string test_str;
    cin >> test_str;

    int current = start_state;
    bool possible = true;

    for (char c : test_str) {
        if (transitions[current].count(c)) {
            current = transitions[current][c];
        } else {
            // If the DFA is "complete" this shouldn't happen,
            // but we handle it for robustness.
            possible = false;
            break;
        }
    }
}

cout << "String: " << test_str << endl;

// Check if final state is a complement final state
bool is_final_in_complement = true;
if (original_final_states.find(current) != original_final_states.end()) {
    is_final_in_complement = false;
}

if (possible && is_final_in_complement) {
    cout << "Accepted" << endl;
} else {
    cout << "Rejected" << endl;
}

```

```
        cout << endl;  
    }  
  
    return 0;  
}
```

Input file ss:

```
"E:\Downloads\Compiler d... X + v - □
3 2
a b
0 a 1
0 b 0
1 a 1
1 b 2
2 a 1
2 b 0
0
1
2 4
Complement Final States:
0 1

ab
String: ab
Rejected

a
String: a
Accepted

bbb aab
String: bbb
Accepted

String: aab
Rejected

Process returned 0 (0x0)   execution time : 17
.790 s
Press any key to continue.
|
```

Task 2:

Write a C++ program that takes two complete DFAs over the same alphabet and constructs a **product DFA**.

The product DFA state will be represented as a pair:

$(state_from_DFA1, state_from_DFA2)$

Your program should support two operations:

- union
- intersection

For union, a product state is final if at least one of the two states is final. For intersection, a product state is final only if both states are final.

Input Format

First, take the alphabet:

- First line: integer m , the number of input symbols
- Next m lines: input symbols

Then take DFA 1:

- number of states n_1
- $n_1 \times m$ transition lines in this format:
current_state input_symbol next_state
- start state of DFA 1
- number of final states of DFA 1
- final states of DFA 1

Then take DFA 2:

- number of states n_2
- $n_2 \times m$ transition lines in this format:
current_state input_symbol next_state
- start state of DFA 2
- number of final states of DFA 2
- final states of DFA 2

Then take:

- operation name: union or intersection
- integer q , the number of test strings
- next q lines, one input string each

Output Format

Print:

- start state of the product DFA
- final states of the product DFA
- product DFA transition table
- result for each input string

Sample Input	Sample Output
2 a b 2 0 a 1 o b o 1 a 1 1 b o o 1 1 2 0 a o o b 1 1 a 1 1 b o o 1 o intersection 4 a ba bba abb	Product DFA Start State: (0,0) Product DFA Final States: (1,0) Product DFA Transition Table: (0,0) --a--> (1,0) (0,0) --b--> (0,1) (0,1) --a--> (1,1) (0,1) --b--> (0,0) (1,0) --a--> (1,0) (1,0) --b--> (0,1) (1,1) --a--> (1,1) (1,1) --b--> (0,0) String: a Accepted String: ba Rejected String: bba Accepted String: abb Rejected

Code:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
#include <map>
```

```
#include <set>
```

```
using namespace std;
```

```
struct DFA {
```

```
    int n;
```

```
    map<pair<int, char>, int> transitions;
```

```
    int start_state;
```

```
    set<int> final_states;
```

```
};
```

```
int main() {
```

```
    int m;
```

```
    cin >> m;
```

```
    vector<char> alphabet(m);
```

```
for (int i = 0; i < m; ++i) cin >> alphabet[i];
```

```
DFA d1;
```

```
cin >> d1.n;
```

```
for (int i = 0; i < d1.n * m; ++i) {
```

```
    int u, v;
```

```
    char c;
```

```
    cin >> u >> c >> v;
```

```
    d1.transitions[{u, c}] = v;
```

```
}
```

```
cin >> d1.start_state;
```

```
int f1;
```

```
cin >> f1;
```

```
for (int i = 0; i < f1; ++i) {
```

```
    int s; cin >> s;
```

```
    d1.final_states.insert(s);
```

```
}
```

```
DFA d2;
```

```
cin >> d2.n;
```

```
for (int i = 0; i < d2.n * m; ++i) {
```

```
int u, v;

char c;

cin >> u >> c >> v;

d2.transitions[{u, c}] = v;
}

cin >> d2.start_state;

int f2;

cin >> f2;

for (int i = 0; i < f2; ++i) {

    int s; cin >> s;

    d2.final_states.insert(s);

}


string op;

cin >> op;

int q;

cin >> q;

vector<string> test_strings(q);

for (int i = 0; i < q; ++i) cin >> test_strings[i];


cout << "Product DFA Start State:" << endl;
```

```
cout << "(" << d1.start_state << "," << d2.start_state << ")" << "\n" << endl;
```

```
cout << "Product DFA Final States:" << endl;
```

```
bool first_f = true;
```

```
for (int i = 0; i < d1.n; ++i) {
```

```
    for (int j = 0; j < d2.n; ++j) {
```

```
        bool is_f1 = d1.final_states.count(i);
```

```
        bool is_f2 = d2.final_states.count(j);
```

```
        bool is_final = (op == "union") ? (is_f1 || is_f2) : (is_f1 && is_f2);
```

```
        if (is_final) {
```

```
            if (!first_f) cout << " ";
```

```
            cout << "(" << i << "," << j << ")";
```

```
            first_f = false;
```

```
        }
```

```
    }
```

```
}
```

```
cout << "\n" << endl;
```

```
cout << "Product DFA Transition Table:" << endl;
```

```

for (int i = 0; i < d1.n; ++i) {
    for (int j = 0; j < d2.n; ++j) {
        for (char c : alphabet) {
            int next1 = d1.transitions[{i, c}];
            int next2 = d2.transitions[{j, c}];

            cout << "(" << i << ", " << j << ") --" << c << "--> (" << next1 << ", "
<< next2 << ")" << endl;

        }
    }
}

cout << endl;

```

```

for (const string& s : test_strings) {
    int curr1 = d1.start_state;
    int curr2 = d2.start_state;

    for (char c : s) {
        curr1 = d1.transitions[{curr1, c}];
        curr2 = d2.transitions[{curr2, c}];
    }

    bool is_f1 = d1.final_states.count(curr1);

```

```
bool is_f2 = d2.final_states.count(curr2);  
bool accepted = (op == "union") ? (is_f1 || is_f2) : (is_f1 && is_f2);  
  
cout << "String: " << s << endl;  
cout << (accepted ? "Accepted" : "Rejected") << "\n" << endl;  
}  
  
return 0;  
}
```

Output:

```
"E:\Downloads\Compiler desi
2 a
b
2
0 a 1
0 b 0
1 a 1
1 b 0
0
1
1
2
0 a 0
0 b 1
1 a 1
1 b 0
0
1
0
intersection
4 a
ba
bba
abb
Product DFA Start State:
(0,0)

Product DFA Final States:
(1,0)

Product DFA Transition Table:
(0,0) --a--> (1,0)
(0,0) --b--> (0,1)
(0,1) --a--> (1,1)
(0,1) --b--> (0,0)
(1,0) --a--> (1,0)
(1,0) --b--> (0,1)
(1,1) --a--> (1,1)
(1,1) --b--> (0,0)

String: a
Accepted
```


String: a
Accepted

String: ba
Rejected

String: bba
Accepted

String: abb
Rejected

Process returned 0 (0x0) execution time : 140.9
s
Press any key to continue.

////////////////////////////////////
End////////////////////////////////////